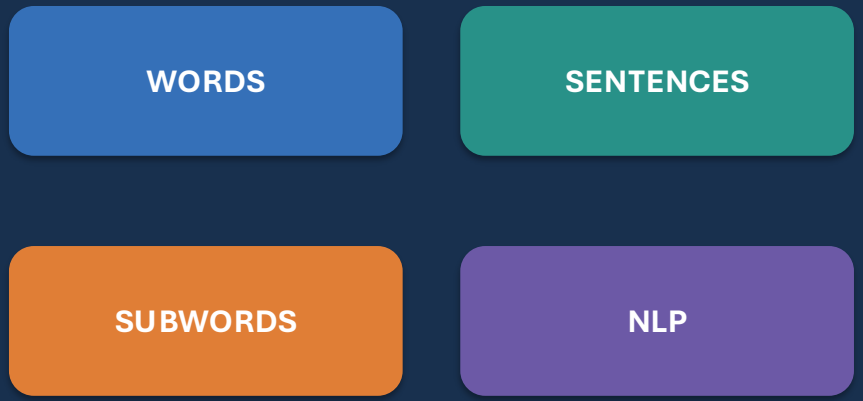
NATURAL LANGUAGE PROCESSING

# Tokenization

From raw text to linguistically meaningful units




Tokenization is a foundational NLP step: it transforms continuous text into units that algorithms can process.

# What Is Tokenization?

Tokenization is the process of segmenting a text into smaller linguistic units called tokens.

### Word tokenization

Splits text into individual word-like units.

Example:
"Natural language processing" → Natural | language | processing

### Sentence tokenization

Splits a document into individual sentences.

Example:
Sentence 1 | Sentence 2 | Sentence 3

### Why it matters

Once text has been segmented, each token can be analyzed for its form, linguistic role, and other properties.

**In modern NLP, the notion of a token may also refer to subword units used by neural language models; this chapter focuses on the word/sentence tokenization presented in the course material.**

# Word Tokenization vs. Simple Splitting

A simple split operation is typically driven by a delimiter such as whitespace. Tokenization is more linguistically informed.

### String splitting

The split operation separates text according to an explicit separator.

It does not inherently understand that punctuation, contractions, or attached symbols may represent distinct linguistic units.

### Tokenization

A tokenizer uses rules or a statistical/model-based strategy to determine meaningful token boundaries.

This becomes important when characters are adjacent but play different linguistic roles.

**Typical challenges include contractions, punctuation, abbreviations, and words that are written without spaces.**

# Challenges in Word Tokenization

### Separating distinct units

A tokenizer should distinguish units such as a contraction or attached punctuation when they represent separate tokens.

### Preserving meaningful compounds

Some sequences should remain together when they function as a single lexical unit.

### Language-specific behavior

Tokenization rules differ across languages. French contractions, German compounds, and scripts without spaces create different segmentation problems.

**It is concerned with splitting a sentence into individual words, and several points should be taken into consideration:**

The algorithm should generally be able to separate different words from one another. For example, I'**d** can sometimes be transformed into I+'**d**.

It should also be able to **combine words that belong together**. For example, **los angeles** is considered **one word/token**, not two.

Some sounds may disappear in certain cases. In French, **L'** can be transformed into **un**.

- **L'ensemble → one token or two?**

  **L ? L' ? Le ?**

  Want **l'ensemble** to match with **un ensemble**.

### French example

In this ex. We highlighted the ambiguity around forms such as "L'ensemble" and whether the apostrophe should define a token boundary.

# Language-Specific Tokenization

Tokenization cannot be treated as a single universal rule. The written conventions of each language influence the correct segmentation.

**Chinese and Japanese**

Japanese and Chinese traditionally do not use spaces between words, making whitespace-based tokenization insufficient.

**Therefore, we use the Max-Match method.**

**Max-Match** means: we start from the beginning of the sentence and identify the **longest sequence of characters that forms a meaningful word**. Once we find it, we start again from the next character and repeat the process.

**Example:**

- **涉拉波娃**现在居住在美国东南部的佛罗里达。
- **涉拉波娃** 现在 **居住 在 美国** 东南部 **的 佛**罗里达
- *Sharapova now lives in US southeastern Florida*

**German Noun Compounds**

German noun compounds are not segmented.

- **Lebensversicherungsgesellschaftsangestellter**
- *life insurance company employee*

This illustrates that a single German compound word may correspond to several words in English.

## German compounds

German can contain very long compound nouns without spaces.

Example:
Lebensversicherungsgesellschaftsangestellter

A tokenizer must decide whether and how such compounds should be segmented.

# Maximum-Match Segmentation

For languages in which word boundaries are not explicitly marked by spaces, a maximum-match strategy can be used.

## Procedure

1. Start at the beginning of the sentence.
2. Identify the longest sequence that corresponds to a meaningful word.
3. Commit to that segment.
4. Continue from the next character.
5. Repeat until the sentence is segmented.

## Intuition

The method searches for the longest plausible lexical unit at each position.

It can work well for languages without spaces, but the choice of dictionary and matching strategy strongly affects the result.

**its effectiveness across English, Chinese, and Japanese.**

# Maximum-Match Segmentation

The following sentence is segmented as follows:

**Thecatinthehat → the cat in the hat**

However, this approach does not always work well in English because the segmentation can produce incorrect results.

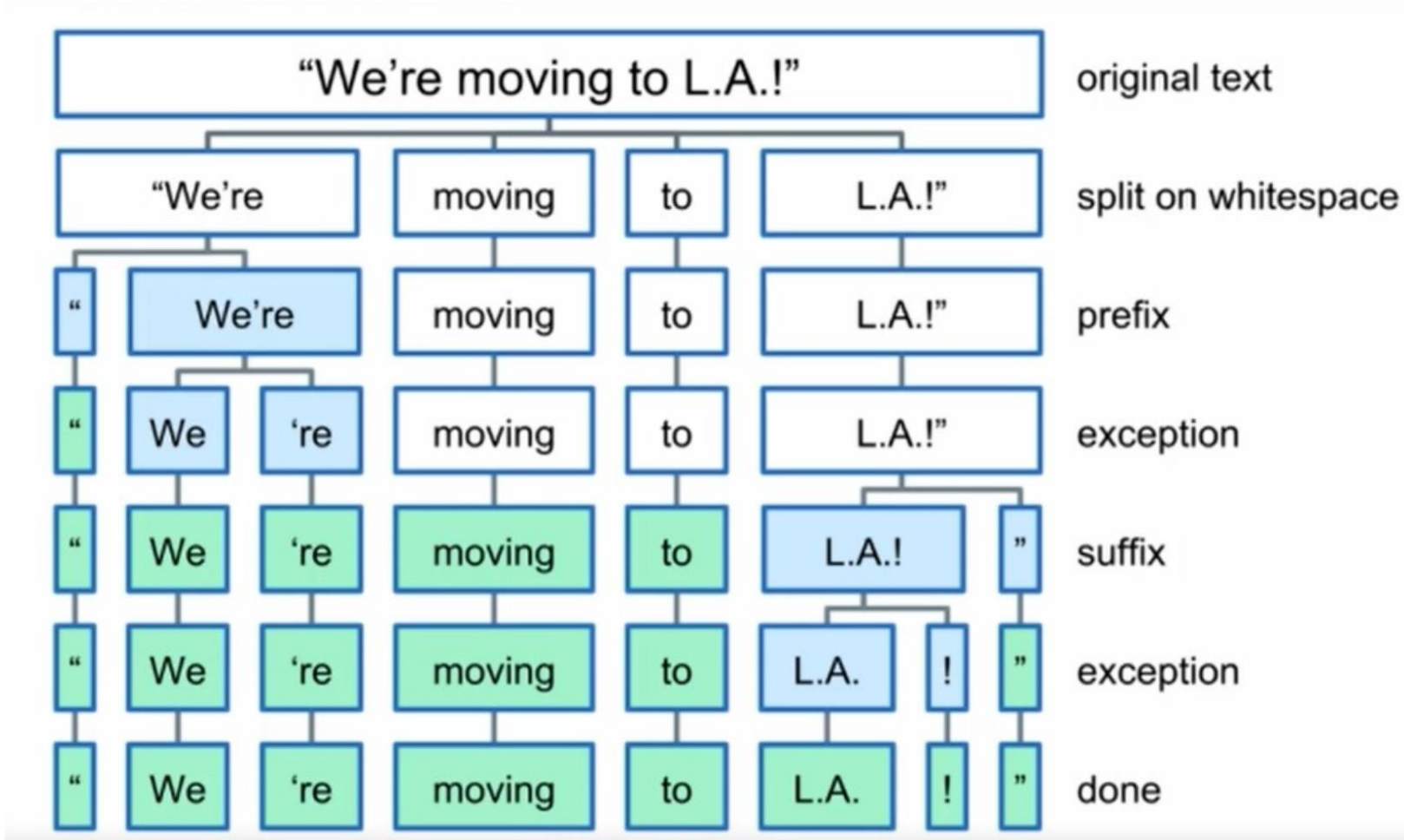
**Thetabledownthere → the table down there**
**thetabledownthere → theta bled own there**

But it can produce positive results in **Chinese and Japanese**.

# A Multi-Stage Tokenization Process

The process is carried out **in several stages**. For example, consider the sentence:
**“We’re moving to L.A.!”**
It is segmented progressively:
**Original text**
“We’re moving to L.A.!”
**Split on whitespace**
→ “We’re” | “moving” | “to” | “L.A.!”
**Prefix handling**
→ “We’re” | “moving” | “to” | “L.A.!”
**Exception handling**
→ “We” | “’re” | “moving” | “to” | “L.A.!”
**Suffix handling**
→ “We” | “’re” | “moving” | “to” | “L.A.” | “!”
**Exception handling**
→ Final tokenization according to the tokenizer’s predefined rules.
**The key idea:** tokenization is not necessarily a single operation. A tokenizer can apply a sequence of rules—such as **whitespace splitting, prefix detection, exceptions, and suffix detection**—to progressively transform the original text into individual tokens.

**Whitespace → quotation marks → apostrophes/contractions → punctuation → exceptions**


“We’re moving to L.A.!”
original text
“We’re
moving
to
L.A.!”
split on whitespace
“
We’re
moving
to
L.A.!”
prefix
“
We
‘re
moving
to
L.A.!”
exception
“
We
‘re
moving
to
L.A.!
”
suffix
“
We
‘re
moving
to
L.A.
!
”
exception
“
We
‘re
moving
to
L.A.
!
”
done

# Tokenization Rules Are Context-Dependent

## No fixed universal sequence

The stages used by a tokenizer do not have to occur in the same order for every sentence.

The appropriate decision depends on the structure of the input.

## Exceptions are essential

Abbreviations, contractions, punctuation, URLs, email addresses, and language-specific forms can require specialized rules.

**A good tokenizer therefore combines general rules with exceptions and language-specific knowledge.**

# Practical Tokenization with spaCy

```
import spacy
nlp = spacy.load('en_core_web_sm')

doc = nlp('Tesla is looking at buying U.S. startup for $6
million')

for token in doc:
    print(token.text)
    print(token.shape_)
    print(token.is_alpha)
    print(token.is_stop)
    print('---------------')
```

## What spaCy provides

spaCy applies an NLP pipeline to the input text and produces token objects that expose useful linguistic and orthographic properties.

## Token-level attributes

text → token string
shape_ → orthographic shape
is_alpha → alphabetic characters?
is_stop → stop word?

# Why Model-Based Tokenization Is More Than split()

Tokenization identifies meaningful text units by detecting boundaries that may not be indicated by spaces alone, such as punctuation marks and special text patterns.

```
doc = nlp('Tesla is looking at buying U.S. startup for
$6 million')
for token in doc:
    print(token.text)
```

**Example: $6 million**

Although “$6” appears as a compact form in the sentence, the tokenizer analyzes the components according to its tokenization rules rather than blindly splitting on whitespace.

**Tokens can then be accessed individually: doc[0], doc[1], ...**

# Selecting a Span of Tokens

```
doc2 = nlp('''
Although commonly attributed to John Lennon from his song
"Beautiful Boy",
the phrase "Life is what happens to us while we are making
other plans"
was written by cartoonist Allen Saunders and published in
Reader's Digest in 1957.
''')
```

```
life_quote = doc2[16:30]
print(life_quote)
```

### Token spans

A slice such as doc2[16:30] selects a contiguous span of tokens from the processed document.

# Handling Contractions and Abbreviations

```
mystring = '"We\'re moving to L.A.!"'
print(mystring)
doc3 = nlp(mystring)

for token in doc3:
    print(token.text, end=' | ')
```

**The staged example shows why contractions such as “We’re” and abbreviations such as “L.A.” require tokenization rules that go beyond whitespace splitting.**

# Additional spaCy Tokenization Examples

```
doc = nlp(u"We're here to help! Send snail-mail, email support@oursite.com or visit us at http://www.oursite.com!")
for token in doc:
    print(token)
```

```
doc = nlp(u"A 5km NYC cab ride costs $10.30")
for token in doc:
    print(token)
```

```
doc = nlp(u"Let's visit St. Louis in the U.S. next year.")
for token in doc:
    print(token)
```

# Token Objects Are Not Assigned Like List Elements

```
doc7 = nlp(u'My dinner was horrible.')
doc8 = nlp(u'Your dinner was delicious.')

# Attempt to replace one token
doc7[3] = doc8[3]
```

**Important distinction**

spaCy Token objects are designed primarily for text inspection and linguistic analysis. Individual tokens cannot be modified through direct item-level assignment. To transform text, modify the original string and process it again with spaCy, or use the appropriate document-processing mechanisms.

**Tokenization produces structured linguistic data; it is not equivalent to editing a Python list of strings.**

# Word Tokenization with NLTK

```
from nltk.tokenize import word_tokenize

EXAMPLE_TEXT = """
Hello Mr. Smith, how are you doing today? The weather is
great,
and Python is awesome. The sky is pinkish-blue. You shouldn't
eat cardboard.
"""

print(word_tokenize(EXAMPLE_TEXT))
```

**word_tokenize()**

NLTK provides a dedicated tokenizer that recognizes punctuation, contractions, and other token boundaries rather than simply splitting on spaces.

We can also use a longer literary passage to demonstrate how tokenization works on realistic, continuous prose, including sentence boundaries, punctuation, and word-level segmentation in context.

# split() vs. word_tokenize()

```
for line in EXAMPLE_TEXT.split('\n')[:20]:
    print(line.split()[:10])
    print('-----------------------')
    print(word_tokenize(line)[:10])
```

### split()

Uses whitespace as the primary delimiter.

Fast and simple, but linguistically unaware.

### word_tokenize()

Applies tokenization rules that account for punctuation and word forms.

More appropriate for NLP preprocessing.

**The difference becomes visible when punctuation and contractions are attached to words.**

# Whitespace Splitting vs. NLP Tokenization

Key idea: Splitting text on whitespace is not the same as linguistic tokenization.

Consider the sentence:

Hello, world! NLP isn't difficult.

```
for line in EXAMPLE_TEXT.split('\n')[:20]:
    print(line.split()[:10])
    print('----------------------')
    print(word_tokenize(line)[:10])
```

| Python split() | NLTK word_tokenize() |
|---|---|
| text.split() | word_tokenize(text) |
| Splits at whitespace.<br>['Hello,', 'world!', 'NLP', "isn't", 'difficult.'] | Separates punctuation and handles common contractions.<br>['Hello', ',', 'world', '!', 'NLP', 'is', "n't", 'difficult', '.'] |

Takeaway: split() preserves punctuation attached to words, while NLTK's tokenizer can recognize punctuation as separate tokens.

# Why Does Tokenization Matter?

Tokenization determines how raw text is represented before many NLP tasks.

| NLP task | Why tokenization matters |
|---|---|
| Sentiment analysis | Negations such as isn't can affect sentiment. |
| Text classification | Token boundaries influence the features extracted from text. |
| Information extraction | Punctuation and word boundaries help identify useful patterns. |
| Language modeling | The model processes text as a sequence of tokens. |

- Important distinction
  - split() uses whitespace as its separator.
  - word_tokenize() applies tokenizer rules designed for text.
  - Neither method is universally best: the appropriate tokenizer depends on the language, task, and model.
- Discussion question: Why might treating isn't as one item versus is and n't affect a downstream NLP task?

# NLTK Tokenizer Resources: punkt and punkt_tab

- NLTK (Natural Language Toolkit) is a Python library for natural language processing. Some of its tokenization functions rely on external data resources.
  - **punkt**
    - Pretrained data used by the Punkt sentence tokenizer to identify sentence boundaries, such as the end of a sentence after a period.
  - **punkt_tab**
    - Additional language-specific data used by newer NLTK versions for Punkt tokenization.
  - Key distinction
    - nltk is the Python library.
    - word_tokenize() is the tokenization function.
    - punkt and punkt_tab are supporting data resources, not Python packages installed with pip.

# Downloading the Required Resources

- Before calling the tokenizer, download the necessary data resources.

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab’)
```

- Then tokenize a sentence:

```
from nltk.tokenize import word_tokenize
text = "Dr. Smith teaches NLP. It's interesting!"
tokens = word_tokenize(text)
print(tokens)
```

- The exact output can vary with the tokenizer and NLTK version.
- Why download both?
  - NLTK versions differ in their resource requirements. Newer versions may require punkt_tab in addition to punkt. Downloading both helps prevent missing-resource errors.

# Understanding a LookupError

- If NLTK cannot find a required resource, it may raise a LookupError.
  - Example message:

    ```
    LookupError:
    Resource punkt_tab not found.
    ```

  - How to troubleshoot
    - Read the error message to identify the missing resource.
    - Download the resource using nltk.download().
    - Run the tokenization cell again.

    ```
    import nltk

    for resource in ['punkt', 'punkt_tab']:
        nltk.download(resource)
    ```

- Takeaway: installing NLTK with pip does not necessarily download all the linguistic data required by its functions.

# Tokenization of Arabic Text

The same tokenization concepts can be applied to Arabic text, while respecting Arabic orthography and punctuation.

```
import spacy
nlp = spacy.load('en_core_web_sm')

text = "أبو عبد الله محمد بن موسى الخوارزمي عالم رياضيات وفلك وجغرافيا مسلم."
doc = nlp(text)

for token in doc:
    print(token.text)
    print(token.shape_)
    print(token.is_alpha)
    print(token.is_stop)
```

**Course example**

The course applies the tokenization workflow to a biographical passage about Muhammad ibn Musa al-Khwarizmi, illustrating how dates, personal names, punctuation, and multiword expressions are segmented into tokens.

**We will also demonstrate token indexing and span extraction on Arabic text.**

# Arabic Tokenization with NLTK

```
from nltk.tokenize import sent_tokenize, word_tokenize

EXAMPLE_TEXT = """
أبو عبد الله محمد بن موسى الخوارزمي عالم رياضيات وفلك وجغرافيا مسلم.
يكنى باسم الخوارزمي وأبي جعفر.
وهو غير مؤكد)م 781هـ 164قيل أنه ولد حوالي .
"""

print(word_tokenize(EXAMPLE_TEXT))
```

## Arabic text processing

We can also apply word_tokenize() to Arabic text and inspect the resulting sequence of tokens programmatically. Keep in mind that tokenization identifies text units but does not, by itself, provide Arabic-specific linguistic analysis.

```
for line in EXAMPLE_TEXT.split('\n')[:20]:
    print(line.split()[:10])
    print(word_tokenize(line)[:10])
```

**Comparing the two outputs makes the effect of linguistically informed tokenization explicit.**

SUMMARY

# Tokenization: Key Takeaways

### Definition

Transform text into manageable linguistic units called tokens.

### Word tokenization

Identify meaningful word-like boundaries rather than blindly splitting on spaces.

### Sentence tokenization

Segment continuous text into individual sentences.

### Language dependence

French, German, Chinese, Japanese, Arabic, and English present different segmentation challenges.

### spaCy

Provides token objects with useful attributes such as text, shape_, is_alpha, and is_stop.

### NLTK

Provides practical functions such as word_tokenize() for rule-based tokenization.

**Tokenization is the bridge between raw natural-language text and the structured representation required by downstream NLP algorithms.**